

12 장 학습 과제

과제 1. 중복 없는 사용자 이름 관리

사용자 이름을 입력받아 `std::set<std::string>`에 저장한다. 이미 등록된 이름이면 중복 메시지를 출력하고, 입력이 끝나면 이름을 정렬된 순서로 출력한다.

과제 2. 아이템 수량 집계

아이템 이름 목록을 `std::vector<std::string>`로 제공받아 `std::map<std::string, int>`로 각 이름의 개수를 집계한다. 구조적 바인딩을 사용해 이름과 수량을 출력한다.

과제 3. map 삽입 함수 비교

같은 키를 대상으로 `insert()`, `try_emplace()`, `insert_or_assign()`, `operator[]`를 호출한다. 키가 없을 때와 있을 때 값이 어떻게 달라지는지 출력한다.

과제 4. 점수 범위 검색

학생 점수를 `std::multiset<int>`에 저장하고 `lower_bound()`와 `upper_bound()`를 이용해 80점 이상 90점 이하의 점수만 출력한다. 해당 범위의 원소 수도 계산한다.

과제 5. 사용자 정의 키 해시

서버 번호와 사용자 번호를 멤버로 가진 `PlayerKey` 구조체를 정의한다. `operator==`와 해시 함수 객체를 작성하고 `std::unordered_map<PlayerKey, std::string, PlayerKeyHash>`에 사용자 이름을 저장한다.

과제 6. 해시 테이블 상태 확인

`std::unordered_set<int>`에 원소를 단계적으로 삽입하면서 `size()`, `bucket_count()`, `load_factor()`를 출력한다. `reserve()` 호출 전후의 버킷 수와 반복자 유효성을 비교한다.

과제 7. 명령 실행 취소

문자열 명령을 `std::stack<std::string>`에 저장하고 사용자가 실행 취소를 요청할 때 가장 최근 명령을 제거한다. 스택이 비어 있을 때는 실행 취소할 명령이 없다는 메시지를 출력한다.

과제 8. 작업 대기열

작업 번호와 설명을 가진 `Job` 구조체를 정의하고 `std::queue<Job>`에 저장한다. 입력 순서대로 작업을 꺼내 처리 결과를 출력한다.

과제 9. 우선순위 작업 처리

작업 번호, 우선순위, 등록 순서를 가진 `Task` 구조체를 정의한다. 우선순위가 높은 작업을 먼저 처리하고 우선순위가 같으면 먼저 등록된 작업을 처리하도록 `std::priority_queue`의 비교 함수 객체를 작성한다.

과제 10. 다형 객체 관리

`GameObject` 추상 클래스와 `Player`, `Monster` 파생 클래스를 정의한다. `std::map<int, std::unique_ptr<GameObject>>`에 객체를 저장하고 식별자 순서로 `Update()`를 호출한다.

11. 반복자 무효화 확인

`std::map`과 `std::unordered_map`에서 원소 반복자와 값 참조를 저장한다. 원소 삽입과 `reserve()`를 수행한 뒤 어떤 위치가 유지되고 어떤 위치가 무효화되는지 주석으로 설명한다. 무효화된 반복자를 실제로 역참조하지 않는다.

12. 컨테이너 선택 설명

제시한 상황마다 적절한 컨테이너를 선택하고 이유를 작성한다.

- 회원 번호로 회원 정보를 빠르게 검색한다.
- 점수를 내림차순으로 유지하며 같은 점수를 허용한다.
- 네트워크 메시지를 도착 순서대로 처리한다.
- 예약 작업을 우선순위와 등록 순서에 따라 처리한다.
- 객체 이름을 중복 없이 저장하고 사전순으로 출력한다.